МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Львівська політехніка»
Інститут комп’ютерних наук та інформаційних технологій
Кафедра «Системи штучного інтелекту»



ЛАБОРАТОРНА РОБОТА №4

з дисципліни
«Методологія та процеси моделювання у штучному інтелекті»

на тему
Побудова та навчання нейронної мережі

Виконав:
студент групи КНСШ-13
Галич Максим

Викладач:
Косаревич Р.Я.

Львів — 2026

Мета роботи. Оволодіти методами проєктування спеціалізованих нейромережевих архітектур (зокрема автоенкодерів та екстракторів ознак) для задач детекції неоднорідностей і аномалій у режимі unsupervised / self-supervised, а також дослідити компроміс між точністю та обчислювальною складністю моделей.

Усі обчислення виконано мовою Python із бібліотеками PyTorch [13], timm [14] та Anomalib [5]. Код продовжує пакет `mk1` з лабораторних робіт №1–3. Завантаження й розбиття даних описано в модулі `mvtec.py`, вилучення ознак — у `patches.py`, запропоновану модель та її навчання — у `feature_ae.py`, дослідження гіперпараметрів — у `ae_search.py`, метрики якості — у `anomaly_metrics.py`, вимірювання обчислювальної складності — у `profiling.py`, порівняння з SOTA-моделями — у `benchmark.py`. Сценарій `lab4/run.py` лише запускає три етапи (дані, пошук гіперпараметрів, порівняння) і перевіряє, що кожен записав свої рисунки та метрики. Увесь експеримент відтворюється однією командою з фіксованим зерном генератора випадкових чисел.

1. ДАНІ ТА ЇХ ПІДГОТОВКА

1.1. Набір даних

Використано набір **MVTec AD** [1] — еталонний набір для детекції аномалій на промислових зображеннях без учителя. Його навчальна частина містить лише зображення без дефектів, а тестова — зображення без дефектів і з дефектами, причому для кожного дефекту є попіксельна маска. Отже, модель навчається тільки на нормі й має знайти те, чого ніколи не бачила, а маски дають змогу оцінити не лише, *чи* є дефект, а й *де* він.

Обрано категорію **bottle** (пляшка, вигляд згори): найпростішу категорію набору, з одним об’єктом у центрі кадру на однорідному тлі. Вона має три типи дефектів: великий скол, малий скол і забруднення (рис. 1.1). Зображення мають розмір 900×900 пікселів, формат RGB.

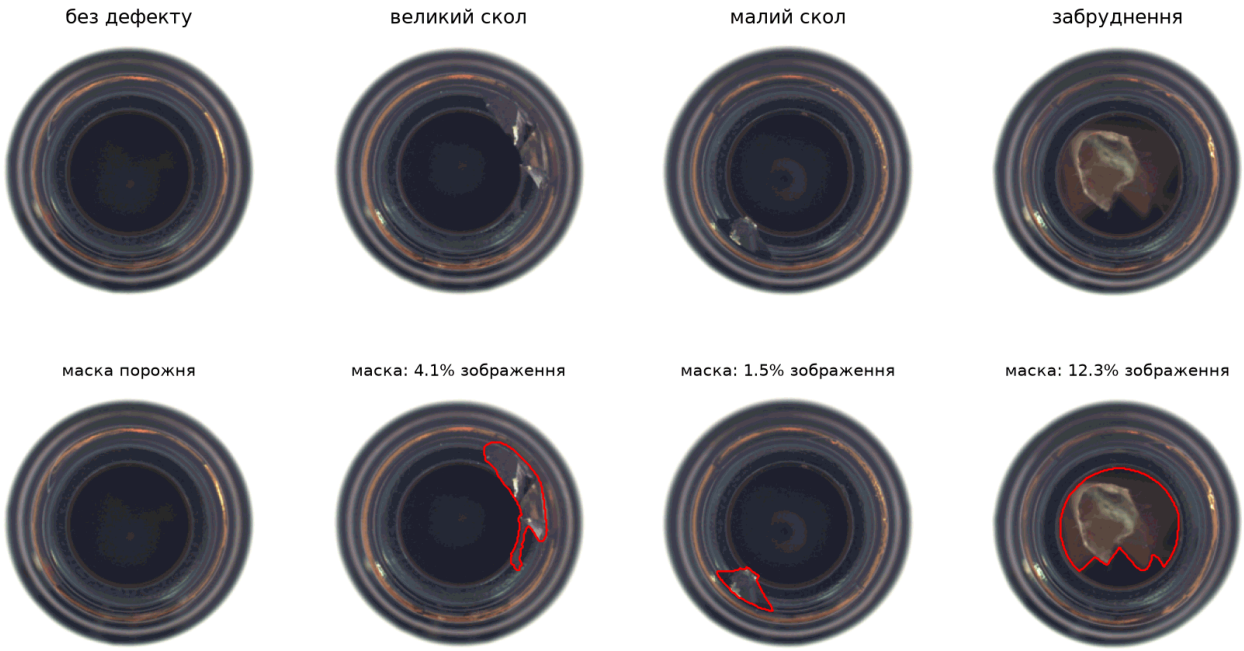


Рис. 1.1. Приклад зображення кожного типу з валідаційної або тестової частини. Унизу червоним обведено маску дефекту і вказано її площу

Завдання вимагає складного набору з високою дисперсією тла або незбалансованістю цілей. Категорія bottle має однорідне тло, але сильно незбалансовані цілі на рівні пікселів (розділ 1.4): дефект займає в середньому кілька відсотків зображення, а тому аномальні пікселі становлять меншість навіть серед зображень із дефектами.

Архів набору не завантажується кодом. Офіційний сайт MVTec видає посилання лише після реєстраційної форми, а під час виконання роботи він був недоступний через помилку TLS-з’єднання. Тому категорію bottle взято з дзеркала на Hugging Face (`foersben/mvtec-ad`) під тією самою ліцензією CC BY-NC-SA 4.0. Кількість файлів у кожній папці збігається з офіційною: 209 навчальних зображень, 83 тестових і 63 маски.

1.2. Розбиття на вибірки

Набір MVTec AD не має валідаційної частини. Якщо обирати гіперпараметри за тестовими зображеннями, підсумкова оцінка буде завищена: у лабораторній роботі №3 показано, що найкраща оцінка пошуку містить оптимізм вибору. Тому тестову частину набору поділено навпіл зі стратифікацією за

типом дефекту (лістинг 1). Перша половина — **валідаційна**: за нею обрано всі гіперпараметри, найкращу епоху навчання та пороги для F1-Score. Друга половина — **тестова**: її використано один раз, в остаточному порівнянні моделей (розділ 6). Розподіл зображень наведено в табл. 1.1.

Таблиця 1.1. Кількість зображень у вибірках

Вибірка	Без дефекту	Великий скол	Малий скол	Забруднення	Усього
Навчальна	209	—	—	—	209
Валідаційна	10	10	11	10	41
Тестова	10	10	11	11	42

```python
def split() -> Splits:
    """
    Cut the benchmark's test part into validation and test halves.

    Returns:
        The untouched training part and the two stratified halves.
    """
    train, test = list_samples(locate())
    val, held_out = train_test_split(
        test,
        test_size=1 - VALIDATION_SHARE,
        stratify=[sample.defect for sample in test],
        random_state=SEED,
    )
    return Splits(train=train, val=val, test=held_out)


def load_image(path: Path) -> torch.Tensor:
    """Read an image, resize it to IMAGE_SIZE and normalise it for the backbones."""
    with Image.open(path) as image:
        rgb = image.convert("RGB").resize(
            (IMAGE_SIZE, IMAGE_SIZE), Image.Resampling.BILINEAR
        )
    array = np.asarray(rgb, dtype=np.float32) / 255.0
    normalised = (array - IMAGENET_MEAN) / IMAGENET_STD
    return torch.from_numpy(normalised.astype(np.float32)).permute(2, 0, 1)
```

Лістинг 1. Розбиття тестової частини і завантаження зображення (`src/mk1/mvtec.py`)

1.3. Перетворення зображень

Кожне зображення зменшено з 900×900 до 256×256 пікселів білінійною інтерполяцією і нормалізовано середнім та стандартним відхиленням ImageNet за кожним каналом кольору. Цей розмір і цю нормалізацію використовують статті PaDiM і PatchCore: на зображеннях такого масштабу навчено переднавчені мережі. Маски зменшено до того самого розміру методом найближчого сусіда, бо інтерполяція створила б частково аномальні пікселі, яких у розмітці немає. Однакові перетворення застосовано до всіх моделей, тож порівняння в розділі 6 не залежить від підготовки даних. Аугментацію не застосовано: модель має вивчити саме норму, а повороти чи зміни кольору могли б зробити нормою те, що в цьому наборі є дефектом.

1.4. Незбалансованість цілей

Площу кожного дефекту обчислено як частку пікселів маски від площі зображення (рис. 1.2). Медіана становить 5,7 %, а діапазон — від 0,57 % до 27,4 %. Малі сколи найменші (медіана 2,4 %, від 0,9 % до 8,1 %), великі сколи — найбільші (медіана 11,9 %). Забруднення мають найширший діапазон: від 0,6 % до 16,5 %. Загалом аномальні пікселі становлять 5,95 % валідаційної і 5,61 % тестової вибірки. Отже, на рівні пікселів співвідношення класів близько 1 : 16, і метрика, яка не враховує цю незбалансованість, переоцінює модель (розділ 3).

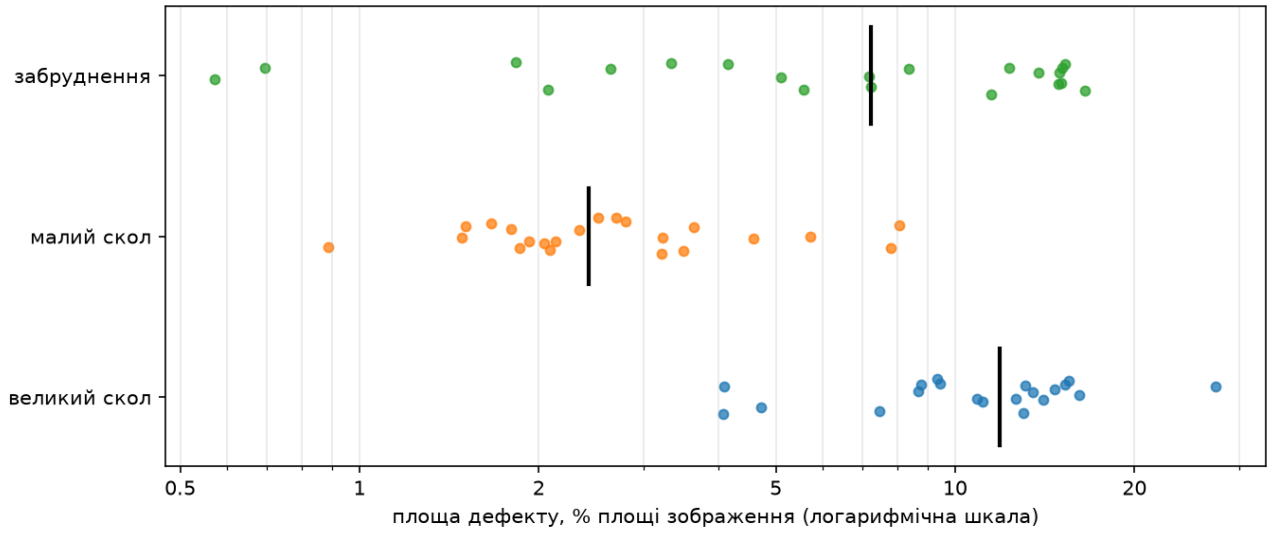


Рис. 1.2. Площа дефекту як частка площі зображення для кожного з 63 дефектних зображень валідаційної і тестової вибірок. Вертикальна риска — медіана типу дефекту

1.5. Розбиття на патчі та вилучення ознак

Ознаки вилучено переднавченими на ImageNet згортковими мережами, ваги яких не змінюються під час навчання. Карта ознак мережі з кроком 8 ділить зображення 256×256 на сітку 32×32 патчі: кожна комірка описує ділянку 8×8 пікселів (рис. 1.3). Як у PatchCore [2], з кожної мережі взято два шари: шар із кроком 8 зберігає локальну текстуру, а шар із кроком 16 додає контекст. Його карту збільшено білінійною інтерполяцією до сітки 32×32 і приєднано до першої за каналами. Порівняно три мережі різної складності (табл. 1.2).

Таблиця 1.2. Мережі-екстрактори ознак

Мережа	**Шари (крок 8 і 16)**	**Каналів**	**Параметрів, млн**	**GFLOPs**
ResNet-18 [7]	`layer2`, `layer3`	128 + 256 = 384	2,78	3,66
WideResNet-50-2 [8]	`layer2`, `layer3`	512 + 1024 = 1536	24,86	24,00
MobileNetV3-Large [9]	`blocks.2`, `blocks.4`	40 + 112 = 152	2,97	0,56

Кількість параметрів і FLOPs виміряно на одне зображення 256×256 для мережі в тому вигляді, у якому її створює бібліотека. Ваги завантажено з репозиторію timm

[14]. ResNet-18 і WideResNet-50-2 timm обрізає після останнього потрібного шару, а MobileNetV3 — ні: вона обчислює всі сім блоків, хоча використовуються виходи лише блоків 2 і 4. Потрібна частина має 0,79 млн параметрів і 0,40 GFLOPs, тож 73 % параметрів і 29 % FLOPs MobileNetV3 витрачаються даремно. Далі наведено виміряні значення, тобто вартість MobileNetV3 у цій роботі завищена.

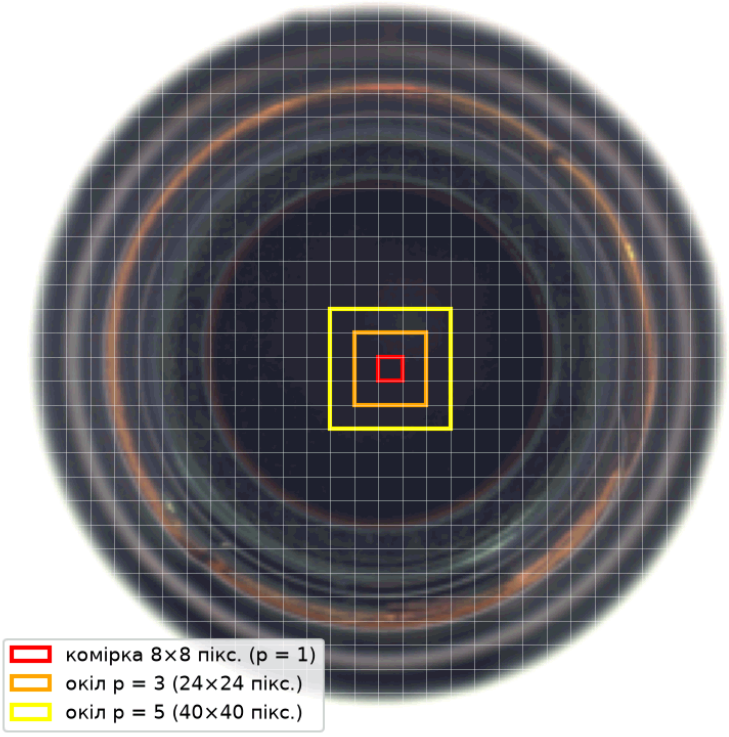


Рис. 1.3. Сітка патчів, яку задає карта ознак із кроком 8: 32×32 ***комірки по*** 8×8 ***пікселів. Кольором показано окіл, за яким усереднюється вектор ознак за розміру патча*** $p = 1$***, 3 і 5***

Розмір патча p задає окіл комірок, за яким усереднюється кожен шар перед об’єднанням (лістинг 2). За $p = 3$ вектор описує ділянку 24×24 пікселі, за $p = 5$ — 40×40 пікселів, без урахування рецептивного поля мережі. Більший окіл стійкіший до шуму і незначних зсувів об’єкта, але розмиває малі дефекти. Під час усереднення крайні комірки враховують лише наявних сусідів (`count_include_pad=False`), щоб нулі доповнення не зменшували ознаки на краю зображення.

```
98 def aggregate(features: list[torch.Tensor], patch_size: int) -> torch.Tensor:
99     """
```

```
    Pool each layer over the patch neighbourhood and stack them on one grid.

    A function rather than a method so that one backbone pass serves every
    patch size of the hyperparameter study.

    Args:
        features: Outputs of PatchEmbedder.layer_features(), stride 8 first.
        patch_size: Side of the neighbourhood, in feature-map cells.

    Returns:
        Embeddings of shape (N, channels, H, W), on the stride-8 grid.
    """
    size = features[0].shape[-2:]
    pooled = []
    for feature in features:
        if patch_size > 1:
            # count_include_pad=False: a border cell averages only the cells
            # that exist instead of being diluted by zero padding.
            feature = F.avg_pool2d(
                feature,
                kernel_size=patch_size,
                stride=1,
                padding=patch_size // 2,
                count_include_pad=False,
            )
        if feature.shape[-2:] != size:
            feature = F.interpolate(
                feature, size=size, mode="bilinear", align_corners=False
            )
        pooled.append(feature)
    return torch.cat(pooled, dim=1)
```

Лістинг 2. Усереднення за околом патча і об’єднання двох шарів (`src/mk1/patches.py`)

Мережа — найдорожча частина обчислень, і вона не змінюється. Тому під час навчання її виходи для всіх зображень обчислено один раз, а усереднення за околом p застосовується до збережених ознак на кожному кроці. Так одне проходження мережі обслуговує всі розміри патча та всі епохи.

2. АРХІТЕКТУРА ЗАПРОПОНОВАНОЇ МОДЕЛІ

2.1. Ідея: автоенкодер ознак

Запропоновано **автоенкодер, що відновлює ознаки патчів**, а не пікселі, у підході DFR (Deep Feature Reconstruction) [4]. Автоенкодер навчається пропускати вектор ознак кожного патча крізь вузьке горло й відновлювати його. Відновлювати він уміє лише те, що бачив, тобто патчі без дефектів. Патч із

дефектом відновлюється погано, і похибка відновлення є оцінкою аномальності. Відновлення ознак замість пікселів — принципове рішення. Піксельний автоенкодер розмиває дрібні деталі, тож похибка відновлення висока і на нормальній текстурі: модель «бачить» аномалії всюди, де є дрібний візерунок. Автоенкодер ознак працює в просторі, де мережа вже відкинула шум та освітлення і зберегла опис вмісту.

2.2. Архітектура

Автоенкодер повністю згортковий (лістинг 3, табл. 2.1). Згортки 1×1 на краях стискають кожен вектор окремо, а згортка 3×3 з кожного боку горла відновлює патч з урахуванням сусідів. Отже, дефект не можна відтворити, просто скопіювавши патч через горло: відновлення спирається на окіл, який здебільшого нормальний. Горло має 64 канали, тобто в 2,4 раза вужче за вхід навіть для MobileNetV3 і в 24 рази — для WideResNet-50-2. Після кожної прихованої згортки йдуть пакетна нормалізація [10] і ReLU.

Таблиця 2.1. Шари автоенкодера (C — кількість каналів ознак, сітка 32×32)

№	Шар	Вхід → вихід	Параметрів за $C = 152$
1	згортка 1×1, BatchNorm, ReLU	$C \to 256$	39 424
2	згортка 3×3, BatchNorm, ReLU	$256 \to 128$	295 168
3	згортка 1×1 (горло)	$128 \to 64$	8 256
4	згортка 1×1, BatchNorm, ReLU	$64 \to 128$	8 448
5	згортка 3×3, BatchNorm, ReLU	$128 \to 256$	295 424
6	згортка 1×1	$256 \to C$	39 064
		Усього	**685 784**

```
87      def __init__(self, channels: int, dropout: float = 0.0) -> None:
88          super().__init__()
89          self.dropout = dropout
90          # Per-channel statistics of the training embeddings. Channels differ in
91          # scale by orders of magnitude, and unstandardised the loss would be
92          # dominated by the few largest.
93          self.register_buffer("mean", torch.zeros(1, channels, 1, 1))
94          self.register_buffer("std", torch.ones(1, channels, 1, 1))
```

```
        self.encoder = nn.Sequential(
            conv_block(channels, HIDDEN, 1),
            conv_block(HIDDEN, HIDDEN // 2, 3),
            nn.Conv2d(HIDDEN // 2, LATENT, 1),
        )
        self.decoder = nn.Sequential(
            conv_block(LATENT, HIDDEN // 2, 1),
            conv_block(HIDDEN // 2, HIDDEN, 3),
            nn.Conv2d(HIDDEN, channels, 1),
        )
```

Лістинг 3. Шари автоенкодера (`src/mk1/feature_ae.py`)

Вхідні ознаки стандартизовано за кожним каналом середнім і стандартним відхиленням навчальних патчів. Канали ознак відрізняються за масштабом на порядки, і без стандартизації функцію втрат визначали б кілька найбільших. Статистики обчислено лише на навчальних зображеннях, як і будь-який крок, що навчається на даних (лабораторна робота №3).

2.3. Оцінка аномальності та регуляризація

Оцінка патча — середньоквадратична похибка відновлення за всіма каналами (лістинг 4). Карта 32×32 збільшується до 256×256 і згладжується гаусовим фільтром із $\sigma = 4$ тим самим кодом, що й у PatchCore з бібліотеки Anomalib. Тому карти двох моделей відрізняються лише способом оцінювання патча. Оцінка зображення — максимум його карти.

Щоб автоенкодер не узагальнював настільки добре, що відновлює й дефекти, використано два регуляризатори:

- **weight decay** — розщеплене згасання ваг оптимізатора AdamW [11];
- **маскувальний шум** — під час навчання частка вхідних ознак обнуляється, і автоенкодер має відновити їх з решти (шумопонижувальний автоенкодер [6]). Значення не масштабуються, як у dropout: обнулені ознаки — це пошкодження, яке треба виправити, а не випадковість, яку слід усереднити.

```
    def forward(self, target: torch.Tensor) -> torch.Tensor:
        """
        Reconstruct standardised embeddings.

        Args:
```

```
            target: Standardised embeddings, (N, channels, H, W).

        Returns:
            The reconstruction, same shape.
        """
        source = target
        if self.training and self.dropout > 0:
            # Masking noise without rescaling: the zeroed features are the
            # corruption to undo, not a stochastic regulariser to average out.
            source = target * (torch.rand_like(target) >= self.dropout)
        return self.decoder(self.encoder(source))

    def patch_scores(self, embedding: torch.Tensor) -> torch.Tensor:
        """
        Reconstruction error of each patch, the anomaly score before upsampling.

        Args:
            embedding: Raw patch embeddings, (N, channels, H, W).

        Returns:
            Mean squared error over the channels, (N, 1, H, W).
        """
        target = self.standardise(embedding)
        return (self(target) - target).pow(2).mean(dim=1, keepdim=True)
```

Лістинг 4. Відновлення з маскувальним шумом і оцінка патча (`src/mk1/feature_ae.py`)

2.4. Навчання

Автоенкодер навчається оптимізатором AdamW із кроком 10^{-3}, функція втрат — середньоквадратична похибка відновлення стандартизованих ознак, 30 епох, пакет із 8 зображень (тобто 8 192 векторів патчів). Після кожної епохи модель оцінюється на валідаційній вибірці двома показниками (лістинг 5):

- **валідаційні втрати** — похибка відновлення на 10 валідаційних зображеннях без дефектів. Вона показує, чи узагальнює автоенкодер норму на зображення, яких не бачив;
- **Pixel_AUROC** на всій валідаційній вибірці — чи відрізняє автоенкодер дефекти від норми.

Повертається модель тієї епохи, на якій валідаційний Pixel_AUROC найвищий, тобто рання зупинка, обрана лише за валідаційною вибіркою.

```
    for epoch in range(1, epochs + 1):
        autoencoder.train()
        order = torch.randperm(images)
        losses = []
```

```
        for start in range(0, images, BATCH):
            batch = select(train_features, order[start : start + BATCH])
            target = autoencoder.standardise(aggregate(batch, config.patch_size))
            loss = F.mse_loss(autoencoder(target), target)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            losses.append(loss.item())

        maps = anomaly_maps(autoencoder, val_features, config.patch_size, image_size)
        record = {
            "epoch": epoch,
            "train_loss": float(np.mean(losses)),
            "val_loss": reconstruction_loss(autoencoder, val_good, config.patch_size),
            "val_pixel_auroc": pixel_auroc(maps, val),
            "seconds": time.time() - started,
        }
        history.append(record)
        log.debug("%s epoch %d: %s", asdict(config), epoch, record)
        if record["val_pixel_auroc"] > best_auroc:
            best_auroc = record["val_pixel_auroc"]
            best_state = copy.deepcopy(autoencoder.state_dict())

    autoencoder.load_state_dict(best_state)
    autoencoder.eval()
    return autoencoder, history
```

Лістинг 5. Цикл навчання з оцінкою після кожної епохи (`src/mk1/feature_ae.py`)

3. МЕТРИКИ ЯКОСТІ ТА ОБЧИСЛЮВАЛЬНОЇ СКЛАДНОСТІ

Кожна модель — і запропонована, і SOTA-моделі з Anomalib — повертає одне й те саме: карту оцінок аномальності, по одній оцінці на піксель. Усі карти оцінюються тим самим кодом (модуль `anomaly_metrics.py`), тож порівняння не залежить від реалізації метрик.

- **Pixel_AUROC** — площа під ROC-кривою за всіма пікселями вибірки відносно масок. Це метрика, яку вимагає завдання і наводять статті про MVTec AD.
- **Pixel PR-AUC** (average precision) — те саме ранжування, оцінене за кривою точність-повнота. Аномальні пікселі становлять 5–6 %, і ROC-AUC мало чутлива до того, скільки нормальних пікселів випереджають аномальні: на кожен аномальний припадає 16 нормальних, і навіть мала частка хибних тривог серед них — це багато хибних

пікселів. PR-AUC, основна метрика лабораторних робіт №1–3, це враховує [12].

- **F1-Score** на рівні пікселів і зображень. F1 потребує порогу, а поріг — це вибір. Його встановлено там, де F1 найвищий на *валідаційній* вибірці, і без змін застосовано до тестової (лістинг 6). Тестовий F1 — це якість детектора, який тестових зображень не бачив.
- **Image_AUROC** — ранжування зображень за максимумом їхньої карти.

```
    val_maps = val_maps.reshape(val.masks.shape)
    test_maps = test_maps.reshape(test.masks.shape)
    pixel_threshold = best_threshold(
        val_maps.flatten().numpy(), val.masks.flatten().numpy()
    )
    image_threshold = best_threshold(image_scores(val_maps), val.anomalous)
    result: dict = {"thresholds": {"pixel": pixel_threshold, "image": image_threshold}}
    for name, maps, part in (("val", val_maps, val), ("test", test_maps, test)):
        result[name] = {
            **ranking_metrics(maps, part),
            "pixel_f1": f1_at(
                maps.flatten().numpy(), part.masks.flatten().numpy(), pixel_threshold
            ),
            "image_f1": f1_at(image_scores(maps), part.anomalous, image_threshold),
        }
    return result
```

Лістинг 6. Пороги F1 з валідаційної вибірки, застосовані до тестової (`src/mk1/anomaly_metrics.py`)

Обчислювальну складність виміряно для детектора повністю, від нормалізованого зображення до карти аномальності, бо саме це оплачує застосування: у PatchCore більшу частину часу займає пошук найближчих сусідів після мережі.

- **FLOPs** пораховано засобом `FlopCounterMode` бібліотеки PyTorch, що реєструє кожне множення матриць і кожну згортку, тож пошук сусідів і відстань Махаланобіса враховано разом із мережами. Одне множення з додаванням — дві операції. Поелементні операції не враховано, вони становлять малу частку в усіх моделях.
- **Затримка** — реальний час одного проходження на CPU з пакетом з одного зображення, як вони надходять на лінії контролю якості: 5 прогрівальних запусків, потім медіана, 10-й і 90-й процентилі за 30

запусками (лістинг 7). Кількість потоків зафіксовано на 8, по одному на фізичне ядро процесора AMD Ryzen 7 8845HS.

GPU, придатного для PyTorch, на комп’ютері немає: вбудовану графіку Radeon 780M використана збірка PyTorch не підтримує. Тому всі вимірювання виконано на CPU. Для кожного детектора окремо виміряно й саму мережу-екстрактор: різниця з повним детектором — вартість того, що працює після мережі.

```
    previous = torch.get_num_threads()
    torch.set_num_threads(THREADS)
    try:
        timings = []
        with torch.no_grad():
            for run in range(WARMUP + REPEATS):
                started = time.perf_counter()
                model(image)
                elapsed = (time.perf_counter() - started) * 1000
                if run >= WARMUP:
                    timings.append(elapsed)
    finally:
        torch.set_num_threads(previous)
    deciles = statistics.quantiles(timings, n=10)
    return {
        "median_ms": statistics.median(timings),
        "p10_ms": deciles[0],
        "p90_ms": deciles[-1],
    }
```

Лістинг 7. Вимірювання затримки на CPU (`src/mk1/profiling.py`)

4. ДОСЛІДЖЕННЯ ГІПЕРПАРАМЕТРІВ

4.1. Схема пошуку

Досліджено чотири гіперпараметри: мережу-екстрактор, розмір патча p і силу обох регуляризаторів. Повний перебір тих самих значень (табл. 4.1) потребував би $3 \cdot 3 \cdot 4 \cdot 3 = 108$ навчань. Тому пошук виконано **по одному фактору**: кожен крок змінює один гіперпараметр, починаючи з переможця попереднього кроку. Такий пошук потребує лише 10 навчань, бо переможець попереднього кроку повторно не навчається. Він припускає, що фактори взаємодіють слабко, тому першим іде фактор, який змінює самі ознаки, — мережа-екстрактор. Початкова конфігурація: ResNet-18, $p = 3$, weight decay 10^{-2} (типове значення AdamW), без маскувального шуму.

Таблиця 4.1. Значення гіперпараметрів у пошуку

Крок	**Гіперпараметр**	**Значення**
1	мережа-екстрактор	ResNet-18, WideResNet-50-2, MobileNetV3-Large
2	розмір патча p	1, 3, 5
3	weight decay	0; 0,01; 0,1; 1
4	частка маскованих ознак	0; 0,2; 0,5

Кожну конфігурацію оцінено за найкращою епохою на валідаційній вибірці. Тестова вибірка на цьому етапі взагалі не завантажується.

4.2. Результати

Результати наведено в табл. 4.2 та на рис. 4.1. Увесь пошук тривав 15 хвилин.

Таблиця 4.2. Результати пошуку на валідаційній вибірці (жирним — переможець кроку)

Конфігурація	**Pixel_ AUROC**	**Pixel PR-AUC**	**Image_ AUROC**	**Найкр. епоха**	**Час, с**
Крок 1: мережа ($p = 3$, weight decay 0,01, без шуму)					
ResNet-18	0,9486	0,3832	0,9935	25	83
WideResNet-50-2	0,9779	0,6256	0,9968	3	172
MobileNetV3-Large	**0,9848**	**0,7521**	**0,9968**	**4**	**73**
Крок 2: розмір патча (MobileNetV3-Large)					
$p = 1$	0,9832	0,7301	1,0000	8	67
$p = 3$	**0,9848**	**0,7521**	**0,9968**	**4**	**73**
$p = 5$	0,9753	0,6517	1,0000	3	68
Крок 3: weight decay ($p = 3$)					
0	**0,9848**	**0,7521**	**0,9968**	**4**	**67**
0,01	0,9848	0,7521	0,9968	4	73
0,1	0,9848	0,7559	0,9968	3	68
1	0,9847	0,7561	0,9968	3	67
Крок 4: частка маскованих ознак (weight decay 0)					
0	0,9848	0,7521	0,9968	4	67
0,2	**0,9851**	**0,7610**	**0,9968**	**3**	**71**
0,5	0,9763	0,7010	0,9871	22	74

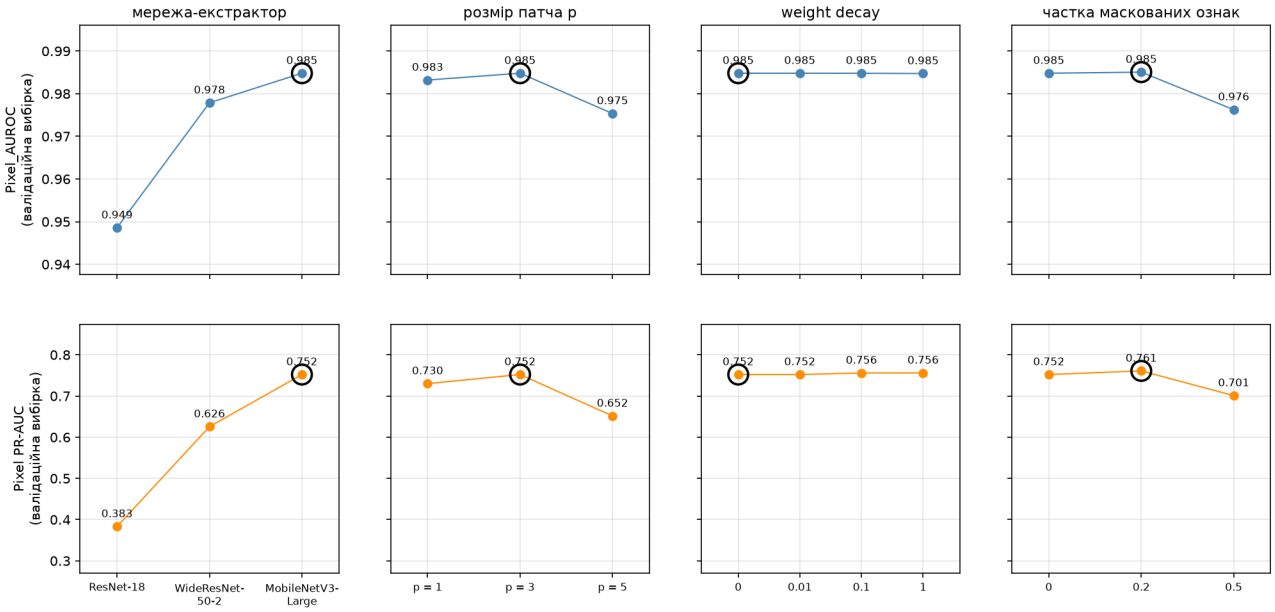


Рис. 4.1. Pixel_AUROC (угорі) і Pixel PR-AUC (унизу) кожної конфігурації на валідаційній вибірці, по стовпцю на крок пошуку. Колом позначено переможця кроку

Мережа-екстрактор — найважливіший фактор. Різниця між мережами (0,036 Pixel_AUROC і 0,37 PR-AUC) більша, ніж сумарний вплив решти трьох гіперпараметрів. Найкращою виявилася найменша мережа, MobileNetV3-Large, хоча вона має у 43 рази менше FLOPs, ніж WideResNet-50-2. Найгіршою — ResNet-18. Автоенкодер на її ознаках повільно покращувався протягом усіх 30 епох (з 0,9385 до 0,9486 на 25-й епосі), але не досяг рівня інших мереж. Причина не в ознаках ResNet-18 як таких: PaDiM і PatchCore на тих самих вагах мають Pixel_AUROC 0,978 (розділ 6). Карти аномальності (рис. 6.1) показують, що автоенкодер на ResNet-18 дуже сильно реагує на великі сколи, але майже не реагує на малі сколи та забруднення. Отже, на цих ознаках похибка відновлення має різний масштаб для різних типів дефектів, і за спільного ранжування пікселів малі дефекти програють великим. Найсильніше це видно за PR-AUC (0,38 проти 0,75).

Розмір патча $p = 3$ **найкращий**, а $p = 5$ — найгірший (0,9753). За медіанної площі 2,4 % малий скол займає близько 40×40 пікселів зображення 256×256, тобто стільки ж, скільки окіл $p = 5$. Половина малих сколів менша, і за $p = 5$ їхні

ознаки усереднюються з нормальним оточенням. Окіл $p = 1$ втрачає небагато (0,9832), але досягає найкращого результату пізніше, на восьмій епосі.

Weight decay майже не впливає на результат. Значення 0 і 0,01 дали однакові оцінки до четвертого знака, 0,1 і 1 — ті самі в межах 0,0001 Pixel_AUROC. Причина в тому, що основним регуляризатором виявилася рання зупинка. Найкраща епоха — 3–4, тобто лише 80–110 кроків оптимізатора. AdamW зменшує ваги на частку $\eta\lambda$ за крок: за $\lambda = 0{,}01$ і кроку $\eta = 10^{-3}$ за 100 кроків ваги зменшуються на 0,1 %, і навіть за $\lambda = 1$ — лише на 10 %. За такий короткий час розщеплене згасання ваг не встигає вплинути. Переможцем кроку формально став $\lambda = 0$, але його перевага над $\lambda = 0{,}01$ — 0,0000015 Pixel_AUROC, тобто випадкова.

Маскувальний шум із часткою 0,2 дав невелике покращення (+0,0003 Pixel_AUROC, +0,009 PR-AUC), а частка 0,5 — погіршення: автоенкодер, якому доводиться відновлювати половину входу, навчається повільно (Pixel_AUROC після першої епохи 0,895 проти 0,983), досягає найкращого результату лише на 22-й епосі і все одно поступається.

Різниці на кроках 3 і 4 (до 0,0004 Pixel_AUROC) того самого порядку, що й коливання між сусідніми епохами одного навчання (рис. 5.1), тож вибір між цими конфігураціями визначає радше шум, ніж реальна перевага. Обрана конфігурація: **MobileNetV3-Large,** $p = 3$**, weight decay 0, частка маскованих ознак 0,2**.

5. НАВЧАННЯ ОБРАНОЇ МОДЕЛІ

Криві навчання обраної конфігурації показано на рис. 5.1, ключові епохи — у табл. 5.1. Одна епоха разом з оцінюванням на валідаційній вибірці триває 2,4 с на CPU, усі 30 епох — 71 с.

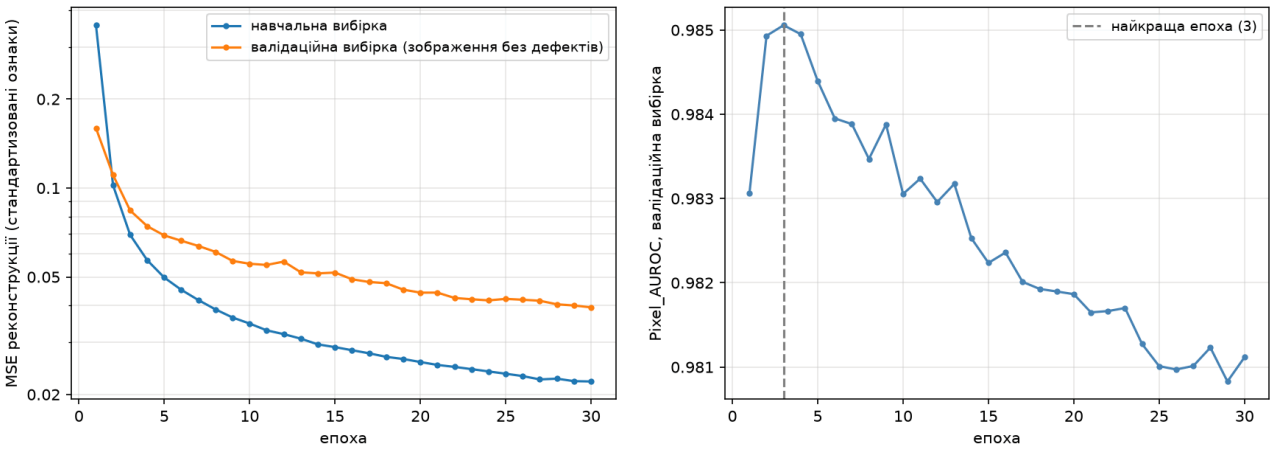


Рис. 5.1. Навчання обраної конфігурації. Ліворуч — втрати відновлення на навчальній вибірці (з маскувальним шумом) і на валідаційних зображеннях без дефектів (без шуму), праворуч — Pixel_AUROC на валідаційній вибірці після кожної епохи

Таблиця 5.1. Втрати й метрика на окремих епохах

Епоха	**1**	**3**	**5**	**10**	**20**	**30**
Втрати, навчальна	0,356	0,069	0,050	0,035	0,026	0,022
Втрати, валідаційна	0,160	0,084	0,069	0,055	0,044	0,039
Pixel_AUROC, валідаційна	0,9831	**0,9851**	0,9844	0,9831	0,9819	0,9811

Втрати зменшуються протягом усього навчання, а якість детекції — ні. Обидві криві втрат спадають монотонно: навчальні — у 16 разів, валідаційні — у 4 рази. Pixel_AUROC досягає максимуму вже на третій епосі (0,9851) і потім повільно знижується до 0,9811 на тридцятій. Це головна особливість автоенкодерів для детекції аномалій: модель навчається не «розпізнавати норму», а відновлювати ознаки, і з кожною епохою відновлює краще *все*, зокрема й дефекти, яких не бачила. Різниця між похибкою на дефекті та на нормі зменшується, і детекція погіршується, хоча за функцією втрат модель стає кращою. Отже, функція втрат не може бути критерієм вибору моделі, і рання зупинка за валідаційною метрикою детекції є необхідною частиною методу.

Починаючи з другої епохи, валідаційні втрати вищі за навчальні, і розрив між ними зростає: з 1,1 раза на другій епосі до 1,8 раза на тридцятій. Отже, автоенкодер дещо перенавчається на навчальних зображеннях, і насправді більше, ніж показують криві: навчальні втрати обчислено з маскувальним шумом, який їх завищує. На першій епосі навчальні втрати вищі за валідаційні, бо вони усереднюються протягом епохи, за яку модель швидко покращується, а валідаційні обчислюються вже після неї.

6. ПРОФІЛЮВАННЯ ТА ПОРІВНЯННЯ З SOTA-МОДЕЛЯМИ

6.1. Моделі для порівняння

Обрану модель порівняно з двома еталонними методами детекції аномалій у реалізації бібліотеки Anomalib [5] з її типовими параметрами:

- **PaDiM** [3] моделює ознаки кожної комірки сітки багатовимірним гаусовим розподілом і оцінює патч відстанню Махаланобіса. Мережа ResNet-18, шари `layer1`–`layer3`, 100 випадково обраних каналів, сітка 64×64;
- **PatchCore** [2] зберігає банк пам’яті з 10 % навчальних патчів, відібраних жадібним алгоритмом coreset, і оцінює патч відстанню до найближчого сусіда в банку. Мережа WideResNet-50-2, як у статті, і ResNet-18 — для порівняння з автоенкодером на тій самій мережі.

PaDiM на WideResNet-50-2 не досліджено: з 550 каналами Anomalib зберігає для кожної з 64×64 комірок матрицю коваріації 550×550, тобто понад 5 ГБ, більше, ніж має вільної пам’яті комп’ютер. Крім того, обрану конфігурацію автоенкодера (з p і регуляризацією з розділу 4) навчено й на двох інших мережах, щоб відокремити внесок мережі від внеску автоенкодера. Усі моделі отримують ті самі зображення, у тому самому розмірі й нормалізації, і оцінюються на тестовій вибірці тим самим кодом. Код навчання обох базових моделей наведено в лістингу 8.

```
130 def fit_padim(train: Part) -> nn.Module:
```

```
    """PaDiM on ResNet-18: a Gaussian per grid cell over 100 random feature channels."""
    torch.manual_seed(SEED)  # the random channel subset
    model = PadimModel(backbone="resnet18", layers=PADIM_LAYERS, pre_trained=True)
    collect(model, train.images)
    model.fit()
    return AnomalibDetector(model)


def fit_patchcore(backbone: str, train: Part) -> nn.Module:
    """PatchCore: a coreset of the training patch embeddings, scored by nearest neighbour."""
    torch.manual_seed(SEED)  # the coreset's random projection and first centre
    model = PatchcoreModel(
        layers=PATCHCORE_LAYERS,
        backbone=backbone,
        pre_trained=True,
        num_neighbors=NEIGHBOURS,
    )
    collect(model, train.images)
    model.subsample_embedding(CORESET_RATIO)
    log.info("PatchCore %s memory bank: %s", backbone, tuple(model.memory_bank.shape))
    return AnomalibDetector(model)
```

Лістинг 8. Навчання PaDiM і PatchCore з бібліотеки Anomalib (`src/mk1/benchmark.py`)

6.2. Якість

Метрики якості на тестовій вибірці наведено в табл. 6.1.

Таблиця 6.1. Якість на тестовій вибірці. Пороги F1 обрано на валідаційній вибірці

Модель	Pixel_ AUROC	Pixel PR-AUC	Pixel F1	Image_ AUROC	Image F1
Автоенкодер (MobileNetV3-Large)	**0,9824**	**0,7193**	**0,7103**	**1,0000**	**0,9841**
Автоенкодер (WideResNet-50-2)	0,9739	0,5640	0,6397	0,9969	0,9841
Автоенкодер (ResNet-18)	0,9470	0,3785	0,4914	1,0000	0,9841
PaDiM (ResNet-18)	0,9776	0,6359	0,6614	0,9969	0,9841
PatchCore (ResNet-18)	0,9782	0,6916	0,6716	1,0000	0,9841
PatchCore (WideResNet-50-2)	0,9848	0,7611	0,7159	1,0000	0,9841

Запропонована модель поступається лише PatchCore на WideResNet-50-2, і то небагато: на 0,0024 Pixel_AUROC, 0,042 PR-AUC і 0,006 Pixel F1. PaDiM і

PatchCore на ResNet-18 вона перевершує за всіма метриками на рівні пікселів: Pixel_AUROC на 0,004–0,005, PR-AUC на 0,03–0,08, F1 на 0,04–0,05.

На рівні зображень задача насичена. Image_AUROC усіх моделей 0,997–1,000, тобто дефектні зображення майже ідеально ранжуються вище за нормальні. Image F1 у всіх однаковий (0,9841 — одне з 32 дефектних зображень нижче порогу): поріг, обраний на 41 валідаційному зображенні, на тестовій вибірці виявився трохи зависоким. Отже, категорія bottle не розрізняє моделі на рівні зображень, і порівнювати їх слід за якістю локалізації дефектів.

Оцінка, за якою обрано модель, оптимістична. На валідаційній вибірці обраний автоенкодер мав Pixel_AUROC 0,9851, на тестовій — 0,9824 (на 0,0027 менше). У PatchCore на WideResNet-50-2, гіперпараметри якого на цих даних не обирали, різниця між двома половинами в 2,5 раза менша (0,0011), у PaDiM її немає зовсім. Імовірно, частина валідаційної оцінки автоенкодера — це оптимізм вибору серед 10 конфігурацій і 30 епох, той самий ефект, що й у лабораторній роботі №3, хоча на 41 і 42 зображеннях різниця такого розміру може бути й випадковою. Саме тому для порівняння використано тестову вибірку, а не валідаційну.

Карти аномальності кожної моделі для одного тестового зображення кожного типу наведено на рис. 6.1.

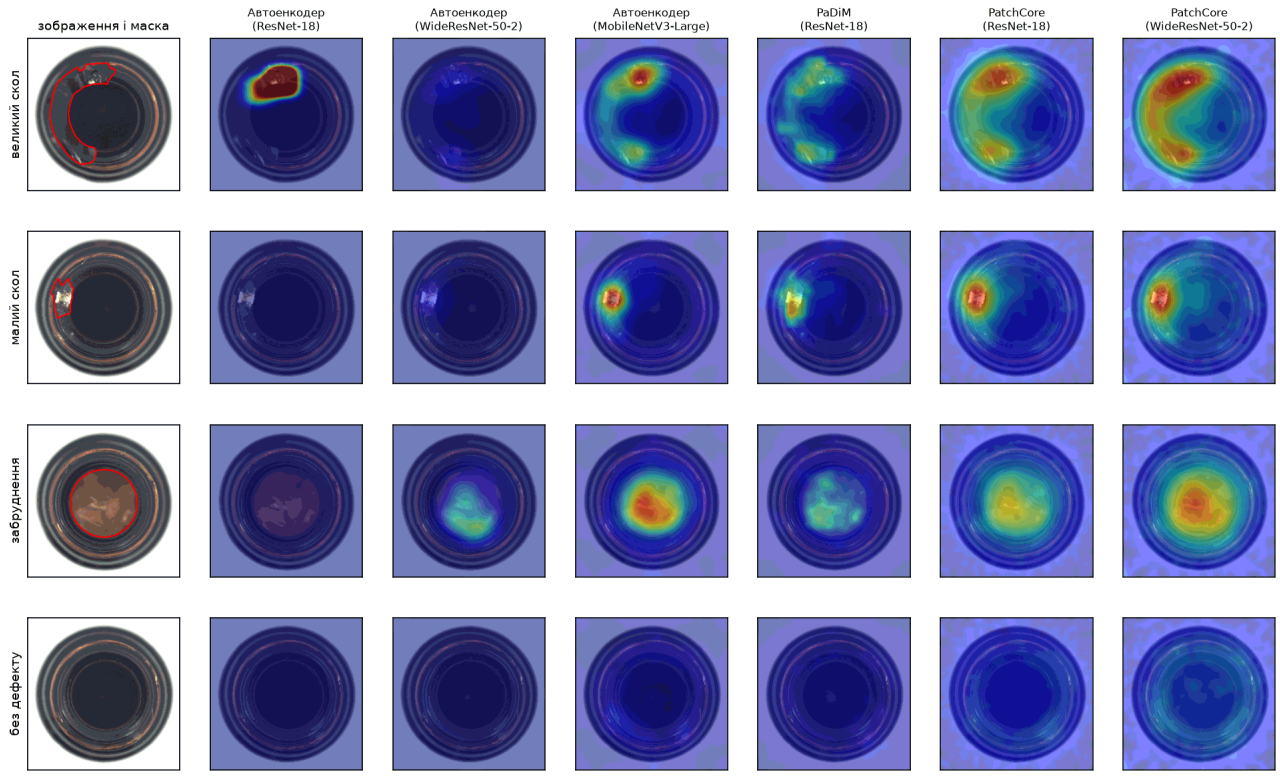


Рис. 6.1. Карти аномальності на тестових зображеннях. Ліворуч — зображення з обведеною маскою дефекту. Шкала кольорів кожної моделі — від 1-го до 99,9-го процентиля її оцінок на всій тестовій вибірці, тож кольори можна порівнювати в межах стовпця

Автоенкодер на MobileNetV3 локалізує всі три типи дефектів, і його карта нормального зображення рівномірно холодна. Карти PatchCore гладші й ширші: на зображенні з великим сколом підвищена оцінка охоплює майже все кільце горлечка навколо дефекту. Автоенкодер на ResNet-18 яскраво виділяє великий скол, але малий скол і забруднення в його шкалі майже не видно — це і є причина його низької PR-AUC.

6.3. Обчислювальна складність

Обчислювальну складність наведено в табл. 6.2 та на рис. 6.2. «Параметри» — ваги мережі-екстрактора та навчені ваги автоенкодера, «пам’ять моделі» — те, що метод зберігає після навчання: гаусові розподіли PaDiM і банк пам’яті PatchCore.

Таблиця 6.2. Обчислювальна складність на CPU (AMD Ryzen 7 8845HS, 8 потоків, пакет з одного зображення 256×256)

Модель	Параметри мережі / голови, млн	Пам'ять моделі, млн чисел	GFLOPs	Затримка, мс/зобр. (p10–p90)	З них мережа, мс	Навчання, с
Автоенкодер (MobileNetV3-Large)	**2,97 / 0,69**	—	**2,10**	**13,8 (13,1–18,4)**	**9,3**	**71**
Автоенкодер (WideResNet-50-2)	24,86 / 1,40	—	26,99	57,0 (51,7–62,7)	42,5	219
Автоенкодер (ResNet-18)	2,78 / 0,81	—	5,45	17,1 (16,3–18,1)	10,0	94
PaDiM (ResNet-18)	2,78 / —	41,4	3,89	39,6 (33,6–47,8)	7,4	8
PatchCore (ResNet-18)	2,78 / —	8,2	20,65	72,7 (71,8–76,5)	7,1	895
PatchCore (WideResNet-50-2)	24,86 / —	32,9	91,53	247,9 (238,5–264,8)	44,2	1025

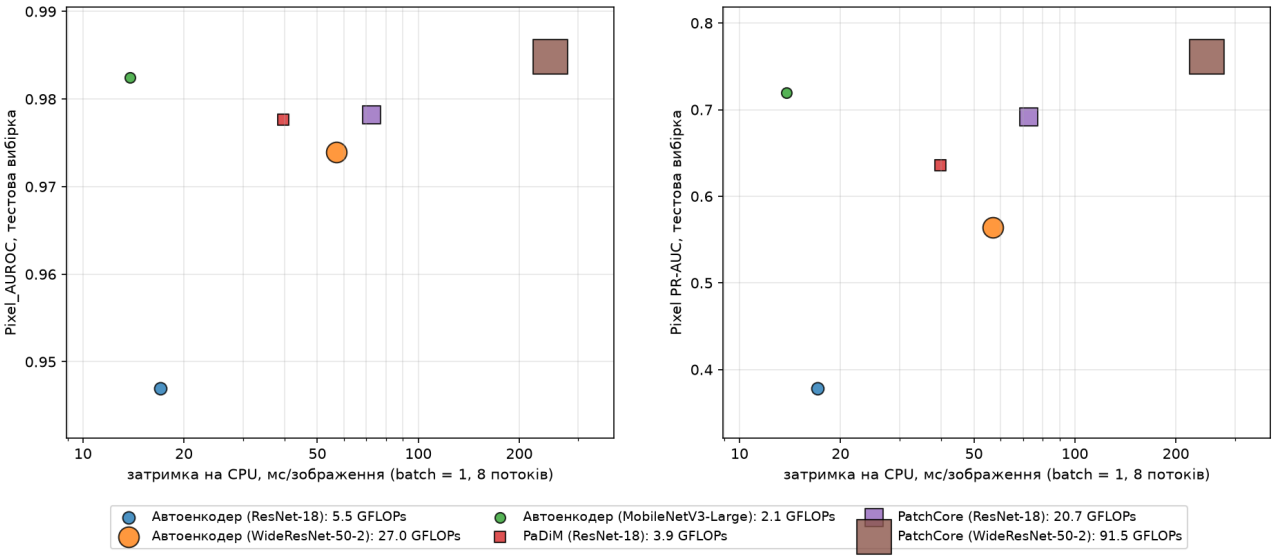


Рис. 6.2. Якість на тестовій вибірці залежно від затримки на CPU (логарифмічна шкала). Площа маркера пропорційна кількості FLOPs; кола — автоенкодер, квадрати — моделі з Anomalib

Запропонована модель у 18 разів швидша за PatchCore на WideResNet-50-2 (13,8 проти 247,9 мс на зображення) і потребує у 44 рази менше обчислень (2,1 проти 91,5 GFLOPs), втрачаючи 0,0024 Pixel_AUROC. Вона також найшвидша з усіх шести моделей: у 2,9 раза швидша за PaDiM і в 5,3 раза — за PatchCore на

ResNet-18, які поступаються їй і за якістю. На рис. 6.2 вона одна займає лівий верхній кут: єдина модель, точніша за неї, повільніша у 18 разів.

Вартість PatchCore визначає не мережа, а пошук сусідів. Мережа WideResNet-50-2 виконує 24 GFLOPs за 44 мс, решта — 67,5 GFLOPs і 204 мс — це порівняння кожного з 1 024 патчів з усіма 21 401 векторами банку пам'яті розмірністю 1 536. Тому заміна мережі на ResNet-18 зменшує FLOPs мережі в 6,6 раза, а повну затримку PatchCore — лише в 3,4 раза: банк пам'яті має ту саму кількість векторів, лише коротших. Вартість пошуку зростає лінійно з кількістю навчальних зображень, тоді як вартість автоенкодера від неї не залежить.

FLOPs не визначають затримку на CPU. MobileNetV3 виконує у 6,5 раза менше операцій, ніж ResNet-18 (0,56 проти 3,66 GFLOPs), але працює не швидше: 9,3 мс проти 7–10 мс. Її глибинні (depthwise) згортки, функції активації hard-swish і блоки squeeze-and-excitation виконують мало операцій на кожен прочитаний байт, тож процесор чекає на пам'ять, а не обчислює: MobileNetV3 виконує близько 60 GFLOP/c, ResNet-18 і WideResNet-50-2 — від 370 до 560. Так само PaDiM виконує лише 0,23 GFLOPs після мережі, але це займає 32 мс: для кожної з 4 096 комірок сітки потрібно прочитати власну обернену матрицю коваріації 100×100, тобто 164 МБ на кожне зображення. Перевага MobileNetV3 у запропонованій моделі полягає не в швидкості мережі, а в тому, що 152 канали її ознак роблять автоенкодер дешевим (1,5 GFLOPs і близько 4,5 мс), і в тому, що на її ознаках автоенкодер найточніший.

Навчання. Автоенкодер навчається 71 с (30 епох), але найкраща епоха третя, тож за ранньої зупинки досить близько 10 с. PaDiM навчається найшвидше (8 с): він лише обчислює середні та коваріації. PatchCore навчається 15–17 хвилин: жадібний вибір 21 401 вектора з 214 016 патчів вимагає 21 401 проходу по всіх патчах.

Пам'ять. Після навчання автоенкодер зберігає 0,69 млн ваг (2,7 МБ), PaDiM — 41,4 млн чисел гаусових розподілів (165 МБ), PatchCore — банк пам'яті з 8,2 або 32,9 млн чисел (33 або 131 МБ). Банк PatchCore і розподіли PaDiM зростають із

роздільністю сітки та, у PatchCore, з кількістю навчальних зображень, а розмір автоенкодера від них не залежить.

7. ВИСНОВКИ

У лабораторній роботі для детекції дефектів на зображеннях без учителя (категорія bottle набору MVTec AD) запропоновано, досліджено й порівняно з SOTA-рішеннями автоенкодер, що відновлює ознаки патчів переднавченої мережі.

1. **Стан даних і виконані перетворення.** Навчальна вибірка — 209 зображень без дефектів, валідаційна і тестова — по половині тестової частини набору (41 і 42 зображення), розбитої зі стратифікацією за трьома типами дефектів. Зображення зменшено з 900×900 до 256×256 пікселів і нормалізовано статистиками ImageNet, маски — зменшено методом найближчого сусіда. Цілі сильно незбалансовані: дефект займає від 0,6 % до 27 % зображення (медіана 5,7 %), аномальні пікселі становлять 5,6–6,0 % вибірки. Тому якість оцінено не лише Pixel_AUROC, а й Pixel PR-AUC. Зображення розбито на сітку 32×32 патчі, кожен з яких описано ознаками двох шарів переднавченої мережі, усередненими за околом $p \times p$ комірок.
2. **Архітектура.** Повністю згортковий автоенкодер із горлом у 64 канали і згортками 3×3 навколо нього відновлює стандартизовані ознаки кожного патча з урахуванням сусідів. Похибка відновлення є оцінкою аномальності. Модель має 0,69 млн навчених параметрів.
3. **Гіперпараметри.** За 10 навчань (замість 108 у повному переборі) обрано MobileNetV3-Large, $p = 3$, без weight decay, з маскуванням 20 % вхідних ознак. Мережа-екстрактор виявилася найважливішим фактором: різниця між мережами 0,036 Pixel_AUROC, більша за вплив решти гіперпараметрів разом. Найкращою була найменша мережа. Окіл $p = 5$ розмиває малі сколи. Weight decay не вплинув на результат, бо

головним регуляризатором стала рання зупинка після 3–4 епох.

4. **Навчання.** Втрати на навчальній і валідаційній вибірках зменшуються протягом усіх 30 епох, а Pixel_AUROC досягає максимуму на третій епосі й потім знижується: автоенкодер учиться відновлювати й дефекти. Отже, функцію втрат не можна використовувати для вибору моделі, а рання зупинка за метрикою детекції на валідаційній вибірці — обов’язкова.
5. **Порівняння з SOTA.** На тестовій вибірці автоенкодер має Pixel_AUROC 0,9824, PR-AUC 0,719 і F1 0,710. Він перевершує PaDiM і PatchCore на ResNet-18 і поступається лише PatchCore на WideResNet-50-2 (0,9848, 0,761 і 0,716). На рівні зображень усі моделі майже ідеальні (Image_AUROC 0,997–1,000), тож категорія bottle розрізняє їх лише за локалізацією дефектів.
6. **Компроміс точності та складності.** Автоенкодер обробляє зображення за 13,8 мс на CPU і виконує 2,1 GFLOPs — у 18 разів швидше й у 44 рази дешевше, ніж PatchCore на WideResNet-50-2 (247,9 мс, 91,5 GFLOPs), за втрати 0,0024 Pixel_AUROC. 82 % часу PatchCore займає пошук найближчих сусідів, і ця частина зростає з обсягом навчальних даних. Затримка на CPU не пропорційна FLOPs: MobileNetV3 має в 6,5 раза менше FLOPs, ніж ResNet-18, але працює не швидше, а PaDiM з найменшою кількістю FLOPs утричі повільніший за автоенкодер, бо читає 164 МБ коваріацій на кожне зображення.

Головний висновок роботи: для детекції дефектів на CPU доцільна архітектура, що спрямовує обчислення на навчену компактну модель, а не на порівняння з усіма навчальними прикладами. Невеликий автоенкодер на ознаках легкої мережі досягає точності, близької до найкращої SOTA-моделі, за частку її вартості, і його вартість не зростає з обсягом даних. Обмеження роботи: одна, найпростіша категорія MVTec AD, валідаційна і тестова вибірки по 41–42 зображення, тож різниці менші за 0,003 Pixel_AUROC не є надійними. Затримку виміряно лише

на CPU.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

[1] Bergmann P., Fauser M., Sattlegger D., Steger C. MVTec AD — A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection // Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2019. P. 9592–9600.

[2] Roth K., Pemula L., Zepeda J., Schölkopf B., Brox T., Gehler P. Towards Total Recall in Industrial Anomaly Detection // Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2022. P. 14318–14328.

[3] Defard T., Setkov A., Loesch A., Audigier R. PaDiM: A Patch Distribution Modeling Framework for Anomaly Detection and Localization // Pattern Recognition. ICPR International Workshops and Challenges. Lecture Notes in Computer Science. 2021. Vol. 12664. P. 475–489.

[4] Shi Y., Yang J., Qi Z. Unsupervised Anomaly Segmentation via Deep Feature Reconstruction // Neurocomputing. 2021. Vol. 424. P. 9–22.

[5] Akcay S., Ameln D., Vaidya A., Lakshmanan B., Ahuja N., Genc U. Anomalib: A Deep Learning Library for Anomaly Detection // Proc. IEEE International Conference on Image Processing (ICIP). 2022. P. 1706–1710.

[6] Vincent P., Larochelle H., Bengio Y., Manzagol P.-A. Extracting and Composing Robust Features with Denoising Autoencoders // Proc. 25th International Conference on Machine Learning (ICML). 2008. P. 1096–1103.

[7] He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition // Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016. P. 770–778.

[8] Zagoruyko S., Komodakis N. Wide Residual Networks // Proc. British Machine Vision Conference (BMVC). 2016. P. 87.1–87.12.

[9] Howard A., Sandler M., Chu G. et al. Searching for MobileNetV3 // Proc. IEEE/CVF International Conference on Computer Vision (ICCV). 2019. P.

1314–1324.

[10] Ioffe S., Szegedy C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift // Proc. 32nd International Conference on Machine Learning (ICML). 2015. P. 448–456.

[11] Loshchilov I., Hutter F. Decoupled Weight Decay Regularization // International Conference on Learning Representations (ICLR). 2019.

[12] Saito T., Rehmsmeier M. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets // PLoS ONE. 2015. Vol. 10, No. 3. e0118432.

[13] Paszke A., Gross S., Massa F. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library // Advances in Neural Information Processing Systems 32. 2019. P. 8024–8035.

[14] Wightman R. PyTorch Image Models [Електронний ресурс]. 2019. URL: https://github.com/huggingface/pytorch-image-models.